Kanban Studio

Usage Manual

A locally hosted project-management application
with AI-powered board assistance

Version: October 2026
Technology: Next.js (frontend) + FastAPI (backend) + SQLite

Table of Contents

1. Introduction and Overview
2. Installation and Setup
3. Starting and Stopping the Application
4. Authentication
5. The Main Board Interface
6. Board Management
7. Column Management
8. Card Management
9. Card Priority and Due Dates
10. Card Checklists
11. AI Chat Assistant
12. Keyboard Shortcuts and Navigation
13. Data Storage and Persistence
14. Architecture Overview
15. API Reference
16. Running Tests
17. Troubleshooting
18. Known Limitations

1. Introduction and Overview

Kanban Studio is a locally hosted project-management application that gives you a visual Kanban board to organize your work. It runs entirely on your machine -- no cloud subscription, no external dependencies for day-to-day use -- with an optional AI chat assistant powered by Groq.

The application is built on two layers: a Next.js frontend that produces a static HTML/CSS/JS export, and a FastAPI backend that serves those files while also hosting the REST API and an SQLite database. In production mode, there is only one server process (the FastAPI backend on port 8000). The Next.js development server (port 3000) is used only during frontend development.

Key Features

- User registration and authentication with secure password hashing (PBKDF2-SHA256)
- Multiple boards per user -- create, rename, switch, and delete boards
- Five fixed, renameable columns per board (Backlog, Discovery, In Progress, Review, Done)
- Cards with title, details, priority (Low/Medium/High), and optional due date
- Per-card checklists with add, toggle, and remove operations
- Drag-and-drop card reordering within and across columns
- AI-powered chat assistant that can create, edit, move cards and rename columns
- Responsive layout with horizontal scrolling on smaller screens
- Real-time error handling with retry capability
- All data stored locally in SQLite -- no external services required for core features

2. Installation and Setup

Prerequisites

- Python 3.12 or later
- Node.js 18 or later (with npm)
- uv (Python package manager) -- install from https://docs.astral.sh/uv/
- Git (for version control)

Backend Setup

From the project root directory, install the Python dependencies:

```
uv sync --project backend
```

This installs FastAPI, Uvicorn, LiteLLM, Pydantic, and all other backend dependencies into an isolated virtual environment managed by uv.

Frontend Setup

From the frontend/ directory, install the Node.js dependencies and build:

```
cd frontend
npm ci
npm run build
```

The build step produces a static export in frontend/out/ which the FastAPI server serves at the root URL. You must rebuild after any frontend code change.

Environment Variables

Create a .env file in the project root with the following optional variable:

```
GROQ_API_KEY=gsk_your_key_here
```

The Groq API key is required only for the AI chat assistant feature. All other features work without it. The key is never exposed to the frontend.

> **Note:** The .env file is git-ignored. Never commit API keys to the repository.

3. Starting and Stopping the Application

Using the Start/Stop Scripts

The project includes convenience scripts for Windows and macOS/Linux:

Windows (PowerShell)

```
.\scripts\start.ps1    # start the server
.\scripts\stop.ps1     # stop the server
```

macOS / Linux

```
sh scripts/start.sh    # start the server
sh scripts/stop.sh     # stop the server
```

The start script launches the FastAPI server on http://127.0.0.1:8000, performs a health check against /api/health, and writes the process ID to .run/server.pid. It refuses to double-start if a server is already running. The stop script reads the PID file and terminates the process.

Manual Start

You can also start the server manually with uv and uvicorn:

```
uv run --project backend --env-file .env \
  uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

Stop with Ctrl+C.

Accessing the Application

Open your browser and navigate to http://127.0.0.1:8000/. If you are not signed in, you will be redirected to the login page at /login/.

4. Authentication

Default Account

On first startup, the application creates a default account with username "user" and password "password". The password is stored as a PBKDF2-SHA256 hash with a random salt -- the plaintext password is never written to the database.

Signing In

1. Navigate to http://127.0.0.1:8000/ (you will be redirected to /login/).
2. Enter your username and password in the form fields.
3. Click "Sign in".

On success, a session cookie is set in your browser and you are redirected to the main board view. The session lasts 8 hours. If you enter incorrect credentials, an error message appears: "Invalid username or password."

Registering a New Account

1. From the login page, click "Create one" (or navigate to /register/).
2. Choose a username (at least 3 characters) and a password (at least 8 characters).
3. Confirm your password by typing it again.
4. Click "Create account".

If the username is already taken, you will see "That username is already taken." On success, you are automatically signed in and a new board called "My Board" is created for you with five empty columns.

Signing Out

Click the "Sign out" button in the top-right corner of the board view. Your session is invalidated on the server and you are redirected to the login page. The cookie is also cleared from your browser.

Note: Sessions are stored in memory. Restarting the backend server signs everyone out.

5. The Main Board Interface

After signing in, you see the main board view. The interface has three main areas:

Header Bar

The header bar at the top contains:

- The "Kanban Studio" brand label
- Board tabs -- one button per board, with the active board highlighted in blue
- "+ New board" button to create additional boards
- "Rename" button to rename the current board
- "Delete" button to delete the current board (disabled if it is your only board)
- A card count badge showing the total number of cards on the current board
- A "Sign out" button

Column Area

Below the header, the board displays five columns in a horizontally scrollable row. Each column has:

- A colored accent bar at the top (yellow, blue, purple, navy, gray)
- An editable column title (click to edit, press Enter or blur to save)
- A card count badge
- The cards belonging to that column
- An "Add a card" button at the bottom

The columns scroll horizontally if the viewport is too narrow to show all five. Each column is 260 pixels wide to ensure card content is readable.

AI Chat Panel

On the right side of the board (or accessible via the floating "Board chat" button on smaller screens), the AI assistant panel provides a conversational interface. You can ask questions about your board or request changes in natural language.

6. Board Management

Each user can have multiple boards. Boards are independent -- each has its own set of five columns and cards.

Creating a New Board

1. Click "+ New board" in the header bar.
2. Type a name for the board (1 to 100 characters).
3. Click "Create board".

The new board is created with five empty columns (Backlog, Discovery, In Progress, Review, Done) and becomes the active board. It appears as a new tab in the header.

Switching Between Boards

Click any board tab in the header bar. The board loads instantly and the column/card area updates to show that board's content.

Renaming a Board

1. Make sure the board you want to rename is the active (selected) board.
2. Click "Rename" in the header bar.
3. Edit the board name in the text field that appears.
4. Click "Save name" to confirm.

Deleting a Board

Click "Delete" in the header bar. The board and all its columns, cards, and checklist items are permanently removed. You cannot delete your only board -- the Delete button is disabled when you have just one board.

Note: Board deletion is immediate and cannot be undone. There is currently no confirmation dialog.

7. Column Management

Each board has exactly five columns. You cannot add, remove, or reorder columns, but you can rename them. The default column names are Backlog, Discovery, In Progress, Review, and Done.

Renaming a Column

1. Click on the column title text at the top of any column.
2. Edit the title in place.
3. Press Enter or click away (blur) to save.

If the save fails (e.g., network error), the column title reverts to its previous value and an error message appears. You can then try again.

Note: Column IDs are stable -- renaming a column changes only its display title, not its internal identifier. The AI assistant uses column IDs, not names, so renamed columns work seamlessly with AI operations.

8. Card Management

Card Anatomy

Each card displays the following information:

- Title -- the card's name, displayed as a bold heading
- Details -- a description of the task, shown below the title
- Priority badge -- a colored label (Low, Medium, or High)
- Due date -- an optional date shown as "Due Oct 15, 2026" format
- Checklist -- a list of sub-items with checkboxes and a progress counter
- Edit button (pencil icon) -- opens the card edit form
- Delete button (trash icon) -- removes the card from the board

Creating a New Card

1. Click "Add a card" at the bottom of the column where you want the card.
2. Fill in the Card title (required, 1-200 characters).
3. Fill in the Details (required, 1-4000 characters).
4. Optionally select a Priority (defaults to Medium).
5. Optionally set a Due date using the date picker.
6. Click "Add card" to create the card.

The "Add card" button is disabled until both the title and details are filled in. The card appears at the bottom of the column immediately after creation.

Editing a Card

1. Click the pencil (edit) icon on the card you want to edit.
2. The card expands into an edit form with fields for title, details, priority, and due date.
3. Make your changes.
4. Click "Save" to apply, or "Cancel" to discard your changes.

While editing, the card cannot be dragged. If the save fails, the card remains in edit mode so you can retry.

Deleting a Card

Click the trash (delete) icon on the card. The card is removed immediately and the remaining cards in that column are reordered. Deletion also removes all checklist items attached to that card.

Moving Cards (Drag and Drop)

Cards can be moved within a column or between columns using drag and drop:

1. Click and hold on a card (the cursor must move at least 6 pixels to start the drag).
2. Drag the card to the desired position in the same or a different column.

3. Release the mouse button to drop the card.

While dragging, the card becomes semi-transparent and a drop indicator highlights the target column. The move is saved to the server immediately on drop. If the save fails, an error message appears.

Note: Drag and drop is disabled while a save operation is in progress or while the AI assistant is processing a request, to prevent conflicting changes.

9. Card Priority and Due Dates

Priority Levels

Each card has a priority level displayed as a colored badge:

- Low -- gray badge, for tasks that are not urgent
- Medium -- blue badge, the default priority for new cards
- High -- purple badge, for urgent or important tasks

Changing a Card's Priority

There are two ways to change a card's priority:

Method 1: Direct Edit

1. Click the pencil (edit) icon on the card.
2. Use the "Priority" dropdown to select Low, Medium, or High.
3. Click "Save".

Method 2: AI Chat

Ask the AI assistant to change the priority. For example:

- "Set the roadmap card to high priority"
- "Make all cards in Review high priority"
- "Change the analytics card priority to low"

Due Dates

Due dates are optional. When set, they appear on the card as a badge in the format "Due Oct 15, 2026". Due dates are stored in YYYY-MM-DD format internally.

Setting a Due Date

1. Edit the card (click the pencil icon).
2. Use the date picker next to "Due date".
3. Click "Save".

Removing a Due Date

Clear the date field in the edit form and save. The due date badge disappears.

10. Card Checklists

Each card can have a checklist -- a list of sub-tasks that can be individually marked as done or not done. Checklists are useful for breaking a card into smaller, actionable steps.

Adding a Checklist Item

1. Find the "Add checklist item" text field at the bottom of any card.
2. Type the text for your checklist item (1-200 characters).
3. Click "Add" or press Enter.

The new item appears in the checklist with an unchecked checkbox. A progress counter appears above the list showing how many items are done (e.g., "Checklist 0/1").

Toggling a Checklist Item

Click the checkbox next to any checklist item to mark it as done (checked) or not done (unchecked). The progress counter updates immediately. The change is saved to the server in the background.

> **Note:** Because the checkbox is a controlled component, it may briefly revert to its previous state until the server confirms the change. This is normal and ensures data consistency.

Removing a Checklist Item

Click the "Remove" button next to any checklist item. The item is deleted immediately and the progress counter updates.

Checklist Persistence

Checklist items are stored in the database and persist across page reloads and server restarts. They are also deleted automatically when the parent card is deleted (cascade delete).

11. AI Chat Assistant

The AI chat assistant lets you interact with your board using natural language. It can answer questions about your board's current state and propose validated changes to cards and columns.

Prerequisites

The AI assistant requires a Groq API key set in the .env file as GROQ_API_KEY. Without this key, the chat will return an error message. All other features of the application work without it.

Opening the Chat Panel

On wide screens (1400px+), the chat panel is always visible on the right side of the board. On smaller screens, click the floating "Board chat" button in the bottom-right corner to open the panel as an overlay.

Sending a Message

1. Type your question or request in the text area at the bottom of the chat panel.
2. Press Enter to send (Shift+Enter for a new line).
3. Or click the "Send" button.

While the AI is processing, a "Thinking..." indicator appears and the board is locked (all editing and drag-and-drop are disabled) to prevent conflicting changes.

What the AI Can Do

The AI assistant supports four types of board operations:

Create Cards

- "Add a new card to Backlog called 'Set up CI pipeline' with details about GitHub Actions"
- "Create a high-priority card in Review for the security audit"

Edit Cards

- "Rename the roadmap card to 'Q4 Roadmap Planning'"
- "Set the analytics card to high priority"
- "Update the details of the onboarding card"

Move Cards

- "Move the analytics prototype to Review"
- "Move all Done cards to Backlog" (if it can identify them)

Rename Columns

- "Rename the first column to 'Ideas'"
- "Change 'Done' to 'Shipped'"

How AI Changes Are Applied

The AI proposes changes as structured operations that are validated by the backend before being applied. The validation checks that:

- Referenced card and column IDs exist on the current board
- The signed-in user owns the board
- All field values meet the schema constraints (non-empty titles, valid priorities, etc.)

All proposed operations are applied in a single database transaction. If any operation fails validation, none are applied -- the board is never left in a partially updated state.

Chat History

The chat panel maintains a conversation history within the current session. Up to 20 previous messages are sent to the AI for context. The history is reset when you reload the page or switch boards.

Error Handling

If the AI request fails (network error, provider error, or invalid response), your question is restored to the text area so you can retry. An error message explains what went wrong.

12. Keyboard Shortcuts and Navigation

Kanban Studio supports the following keyboard interactions:

- Enter in the column title field: saves the column rename
- Enter in the chat text area: sends the message
- Shift+Enter in the chat text area: inserts a new line
- Tab: navigates between interactive elements (standard browser behavior)
- Escape: not currently bound (use Cancel buttons to close forms)

The application uses standard HTML form controls with proper labels and ARIA attributes for screen reader accessibility.

13. Data Storage and Persistence

All application data is stored in a SQLite database file. By default, the database is created at .data/kanban.sqlite3 in the project root on first startup.

Overriding the Database Path

Set the KANBAN_DB_PATH environment variable to use a different location:

```
KANBAN_DB_PATH=/path/to/my/database.sqlite3
```

Database Schema

The database contains five tables:

- schema_migrations -- tracks which migrations have been applied
- users -- user accounts with PBKDF2-hashed passwords
- boards -- boards with title, owner, position, and timestamps
- columns -- the five columns per board with title and position
- cards -- cards with title, details, priority, due date, column assignment, and position
- card_checklist_items -- checklist items with text, done status, and position

Foreign keys enforce referential integrity. Deleting a board cascades to its columns, cards, and checklist items. Deleting a card cascades to its checklist items.

Backups

To back up your data, simply copy the .data/kanban.sqlite3 file while the server is stopped. The database uses WAL mode implicitly through SQLite defaults.

14. Architecture Overview

Single-Server Model

In production mode, there is no Next.js runtime. The frontend is built as a static export (HTML, CSS, JS files in frontend/out/) which the FastAPI server serves at /. The same FastAPI process handles API requests at /api/*. This means you only need to run one server process.

Backend Modules

The backend code is organized into focused modules:

- main.py -- app factory (create_app), lifespan, CORS config, health endpoint, static file serving
- schemas.py -- Pydantic request/response models with field validation
- dependencies.py -- FastAPI dependency injection providers
- auth.py -- password hashing, session token management
- auth_routes.py -- login, register, session check, logout endpoints
- board_routes.py -- board, column, card, and checklist CRUD endpoints
- ai_routes.py -- AI chat endpoint
- ai.py -- LiteLLM/Groq integration, prompt construction
- ai_models.py -- AI request/response schemas with discriminated union for board changes
- database.py -- SQLite schema, migrations, and all database operations

Frontend Components

The frontend is built with React components:

- KanbanBoard -- main board view, state management, drag-and-drop context
- KanbanColumn -- column display with card list and new-card form
- KanbanCard -- individual card display and edit form
- KanbanCardPreview -- drag overlay preview of a card
- CardMeta -- priority badge and due-date display
- CardChecklist -- checklist items with add/toggle/remove
- NewCardForm -- form for creating new cards
- BoardChat -- AI chat panel with conversation history
- BoardBar -- board tab switcher with create/rename/delete
- LoginForm -- sign-in page
- RegisterForm -- registration page

Design Tokens

The application uses five consistent brand colors defined as CSS custom properties:

- Yellow (#ecad0a) -- accent color for drag highlights

- Blue (#209dd7) -- primary interactive color, links, active states
- Purple (#753991) -- buttons, action elements
- Navy (#032147) -- text, headings
- Gray (#888888) -- secondary text, muted elements

15. API Reference

All API endpoints are served under /api/. Authentication is via a session cookie (kanban_session) set on login or registration. Endpoints that require authentication return 401 if the session is missing or expired.

Health

GET	/api/health	Returns {"status": "ok"}. No auth required.

Authentication

POST	/api/auth/login	Sign in. Body: {username, password}. Returns {username}.
POST	/api/auth/register	Create account. Body: {username, password}. Returns 201.
GET	/api/auth/session	Check current session. Returns {username}.
POST	/api/auth/logout	Sign out. Returns 204. Clears session cookie.

Boards

GET	/api/boards	List all boards for the signed-in user.
POST	/api/boards	Create a board. Body: {title}. Returns 201.
GET	/api/boards/{id}	Get a board's full data (columns, cards, checklists).
PATCH	/api/boards/{id}	Rename a board. Body: {title}.
DELETE	/api/boards/{id}	Delete a board. Returns 204 (or 409 if last board).

Columns

PATCH	/api/boards/{id}/columns/{col}	Rename a column. Body: {title}.

Cards

POST	/api/boards/{id}/columns/{col}/cards	Create a card. Body: {title, details, priority?, due_date?}.
PATCH	/api/boards/{id}/cards/{card}	Update a card. Body: {title, details, priority?, due_date?}.
PATCH	/api/boards/{id}/cards/{card}/move	Move a card. Body: {column_id, position}.
DELETE	/api/boards/{id}/cards/{card}	Delete a card and its checklist items.

Checklist Items

POST	/api/boards/{id}/cards/{card}/checklist	Add item. Body: {text}. Returns 201.
PATCH	/api/boards/{id}/cards/{card}/checklist/{item}	Toggle item. Body: {done: true/false}.
DELETE	/api/boards/{id}/cards/{card}/checklist/{item}	Remove checklist item.

AI Chat

POST	/api/boards/{id}/chat	Send a message. Body: {question, history?}. Returns {reply, changed, board}.

16. Running Tests

Backend Tests

```
uv run --project backend pytest                      # all tests
uv run --project backend pytest backend/tests/test_main.py::test_name  # single test
```

The backend has 83 tests covering authentication, board CRUD, card operations, checklist operations, migrations, and AI model validation. One test is skipped by default: the live Groq connectivity test, which requires RUN_GROQ_CONNECTIVITY=1 and a valid GROQ_API_KEY.

Frontend Unit Tests

```
cd frontend
npm run test:unit                  # all unit tests
npx vitest run src/lib/kanban.test.ts  # single file
```

The frontend has 24 unit and component tests covering the KanbanBoard, LoginForm, RegisterForm, and the moveCard helper function.

End-to-End Tests (Playwright)

```
cd frontend
npm run test:e2e                   # all e2e tests
npx playwright test tests/auth.spec.ts  # single spec
```

The e2e suite has 14 tests running in Chromium. It starts the FastAPI server automatically with a temporary database, exercises the full application through the browser, and tears everything down afterwards. Tests cover authentication, board management, card CRUD, column renaming, error handling, checklists, and drag-and-drop.

> **Note:** Port 8000 must be free before running e2e tests. Stop any running dev server first.

Running All Tests

```
cd frontend
npm run test:all    # runs unit tests, then e2e
```

17. Troubleshooting

"Unable to load your boards" on Login

This error appears when the frontend cannot reach the backend API. Check that the server is running on port 8000 and that no firewall is blocking local connections.

Port 8000 Already in Use

If the start script reports that port 8000 is in use, another process may be holding it. Run the stop script first, or find and terminate the process manually.

AI Chat Returns an Error

Common causes:

- GROQ_API_KEY not set or invalid -- check your .env file
- Network connectivity issues -- the Groq API requires internet access
- Rate limiting -- the Groq free tier has request limits; wait and retry
- Invalid AI response -- the model occasionally returns malformed JSON; retry

Session Expired After Server Restart

Sessions are stored in memory and are lost when the server restarts. Simply sign in again. Your board data in SQLite is unaffected.

Frontend Changes Not Showing

The FastAPI server serves the static export from frontend/out/. After changing frontend code, you must rebuild:

```
cd frontend
npm run build
```

Database Corruption

If the database becomes corrupted, delete .data/kanban.sqlite3 and restart the server. A fresh database with the default account and seeded board will be created. Previous data cannot be recovered unless you have a backup.

18. Known Limitations

- Fixed column count: Each board has exactly five columns. You cannot add, remove, or reorder columns.
- No concurrent editing: The application does not synchronize changes between multiple browser tabs or users in real time. The last save wins.
- In-memory sessions: Restarting the server signs out all users. Sessions are not persisted to the database.
- No board sharing: Each user sees only their own boards. There is no collaboration or sharing mechanism.
- No card labels or assignees: Cards have priority and due date but no custom labels, tags, or user assignment.
- No activity log: There is no audit trail of board changes. Undo is not supported.
- AI assistant requires internet: The Groq API call goes over the network. All other features work fully offline.
- No file attachments: Cards cannot have attached files or images.
- Board delete has no confirmation: Clicking Delete immediately removes the board without asking for confirmation.
- No dark mode: The application uses a light theme only. CSS custom properties are in place for a future dark-mode implementation.